1. Loading the data using pandas

[1]:
```
#Load the data file using pandas.
import pandas as pd
data = pd.read_csv('googleplaystore.csv')
```

[2]:
```
data.head()
```

[2]:

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	January 7, 2018	1.0.0	4.0.3 and up
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone	Art & Design;Pretend Play	January 15, 2018	2.0.0	4.0.3 and up
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7M	5,000,000+	Free	0	Everyone	Art & Design	August 1, 2018	1.2.4	4.0.3 and up
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	215644	25M	50,000,000+	Free	0	Teen	Art & Design	June 8, 2018	Varies with device	4.2 and up
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	967	2.8M	100,000+	Free	0	Everyone	Art & Design;Creativity	June 20, 2018	1.1	4.4 and up

2. Checking number of null values in each column

[3]:
```
#Check for null values in the data. Get the number of null values for each column.
data.isna().sum()
```

[3]:
```
App                0
Category           0
Rating          1474
Reviews            0
Size               0
Installs           0
Type               1
Price              0
Content Rating     1
Genres             0
Last Updated       0
Current Ver        8
Android Ver        3
dtype: int64
```

3. Dropping records with nulls

[4]:
```
df = df.dropna()
df.isnull().sum()
```

[4]:
```
App               0
Category          0
Rating            0
Reviews           0
Size              0
Installs          0
Type              0
Price             0
Content Rating    0
Genres            0
Last Updated      0
Current Ver       0
Android Ver       0
dtype: int64
```

[5]:
```
#resetting the index
df = df.reset_index(drop=True)
```

4. Fixing Formatting and Data types

4.1 Fixing Size

Displaying every unique value in size

Fixing the size column

```
[7]: #displaying every unique value in the size column
     print(df['Size'].unique())
```

```
['19M' '14M' '8.7M' '25M' '2.8M' '5.6M' '29M' '33M' '3.1M' '28M' '12M'
 '20M' '21M' '37M' '5.5M' '17M' '39M' '31M' '4.2M' '23M' '6.0M' '6.1M'
 '4.6M' '9.2M' '5.2M' '11M' '24M' 'Varies with device' '9.4M' '15M' '10M'
 '1.2M' '26M' '8.0M' '7.9M' '56M' '57M' '35M' '54M' '201k' '3.6M' '5.7M'
 '8.6M' '2.4M' '27M' '2.7M' '2.5M' '7.0M' '16M' '3.4M' '8.9M' '3.9M'
 '2.9M' '38M' '32M' '5.4M' '18M' '1.1M' '2.2M' '4.5M' '9.8M' '52M' '9.0M'
 '6.7M' '30M' '2.6M' '7.1M' '22M' '6.4M' '3.2M' '8.2M' '4.9M' '9.5M'
 '5.0M' '5.9M' '13M' '73M' '6.8M' '3.5M' '4.0M' '2.3M' '2.1M' '42M' '9.1M'
 '55M' '23k' '7.3M' '6.5M' '1.5M' '7.5M' '51M' '41M' '48M' '8.5M' '46M'
 '8.3M' '4.3M' '4.7M' '3.3M' '40M' '7.8M' '8.8M' '6.6M' '5.1M' '61M' '66M'
 '79k' '8.4M' '3.7M' '118k' '44M' '695k' '1.6M' '6.2M' '53M' '1.4M' '3.0M'
 '7.2M' '5.8M' '3.8M' '9.6M' '45M' '63M' '49M' '77M' '4.4M' '70M' '9.3M'
 '8.1M' '36M' '6.9M' '7.4M' '84M' '97M' '2.0M' '1.9M' '1.8M' '5.3M' '47M'
 '556k' '526k' '76M' '7.6M' '59M' '9.7M' '78M' '72M' '43M' '7.7M' '6.3M'
 '334k' '93M' '65M' '79M' '100M' '58M' '50M' '68M' '64M' '34M' '67M' '60M'
 '94M' '9.9M' '232k' '99M' '624k' '95M' '8.5k' '41k' '292k' '80M' '1.7M'
 '10.0M' '74M' '62M' '69M' '75M' '98M' '85M' '82M' '96M' '87M' '71M' '86M'
 '91M' '81M' '92M' '83M' '88M' '704k' '862k' '899k' '378k' '4.8M' '266k'
 '375k' '1.3M' '975k' '980k' '4.1M' '89M' '696k' '544k' '525k' '920k'
 '779k' '853k' '720k' '713k' '772k' '318k' '58k' '241k' '196k' '857k'
 '51k' '953k' '865k' '251k' '930k' '540k' '313k' '746k' '203k' '26k'
 '314k' '239k' '371k' '220k' '730k' '756k' '91k' '293k' '17k' '74k' '14k'
 '317k' '78k' '924k' '818k' '81k' '939k' '169k' '45k' '965k' '90M' '545k'
 '61k' '283k' '655k' '714k' '93k' '872k' '121k' '322k' '976k' '206k'
 '954k' '444k' '717k' '210k' '609k' '308k' '306k' '175k' '350k' '383k'
 '454k' '1.0M' '70k' '812k' '442k' '842k' '417k' '412k' '459k' '478k'
 '335k' '782k' '721k' '430k' '429k' '192k' '460k' '728k' '496k' '816k'
 '414k' '506k' '887k' '613k' '778k' '683k' '592k' '186k' '840k' '647k'
 '373k' '437k' '598k' '716k' '585k' '982k' '219k' '55k' '323k' '691k'
 '511k' '951k' '963k' '25k' '554k' '351k' '27k' '82k' '208k' '551k' '29k'
 '103k' '116k' '153k' '209k' '499k' '173k' '597k' '809k' '122k' '411k'
 '400k' '801k' '787k' '50k' '643k' '986k' '516k' '837k' '780k' '20k'
 '498k' '600k' '656k' '221k' '228k' '176k' '34k' '259k' '164k' '458k'
 '629k' '28k' '288k' '775k' '785k' '636k' '916k' '994k' '309k' '485k'
 '914k' '903k' '608k' '500k' '54k' '562k' '847k' '948k' '811k' '270k'
 '48k' '523k' '784k' '280k' '24k' '892k' '154k' '18k' '33k' '860k' '364k'
 '387k' '626k' '161k' '879k' '39k' '170k' '141k' '160k' '144k' '143k'
 '190k' '376k' '193k' '473k' '246k' '73k' '253k' '957k' '420k' '72k'
 '404k' '470k' '226k' '240k' '89k' '234k' '257k' '861k' '467k' '676k'
 '552k' '582k' '619k']
```

Replacing “Varies with device”, extracting numeric values, and converting

```
[8]: #Cleaning the Size Column
     def clean_size(x):
         if x == 'Varies with device':
             return np.nan
         elif 'M' in x:
             return float(x.replace('M', '')) * 1000
         elif 'k' in x:
             return float(x.replace('k', ''))
         return np.nan
```

```
[9]: df['Size'] = df['Size'].apply(clean_size)
```

New unique entries in size

```
[10]: print(df['Size'].unique())
```

```
[1.90e+04 1.40e+04 8.70e+03 2.50e+04 2.80e+03 5.60e+03 2.90e+04 3.30e+04
 3.10e+03 2.80e+04 1.20e+04 2.00e+04 2.10e+04 3.70e+04 5.50e+03 1.70e+04
 3.90e+04 3.10e+04 4.20e+03 2.30e+04 6.00e+03 6.10e+03 4.60e+03 9.20e+03
 5.20e+03 1.10e+04 2.60e+04      nan 9.40e+03 1.50e+04 1.00e+04 1.20e+03
 2.60e+04 8.00e+03 7.90e+03 5.60e+04 5.70e+04 3.50e+04 5.40e+04 2.01e+02
 3.60e+03 5.70e+03 8.60e+03 2.40e+03 2.70e+04 2.70e+03 2.50e+03 7.00e+03
 1.60e+04 3.40e+03 8.90e+03 3.90e+03 2.90e+03 3.80e+04 3.20e+04 5.00e+03
 1.80e+04 1.10e+03 2.20e+03 4.50e+03 9.80e+03 5.20e+04 9.00e+03 6.70e+03
 3.00e+04 2.60e+03 7.10e+03 2.20e+04 6.40e+03 3.20e+03 8.20e+03 4.90e+03
 9.50e+03 5.00e+03 5.90e+03 1.30e+04 7.30e+04 6.80e+03 3.50e+03 4.00e+03
 2.30e+03 2.10e+03 4.20e+04 9.10e+03 5.50e+04 2.30e+01 7.30e+03 6.50e+03
 1.50e+03 7.50e+03 5.10e+04 4.10e+04 4.80e+04 8.50e+03 4.60e+04 8.30e+03
 4.30e+03 4.70e+03 3.30e+03 4.00e+04 7.00e+03 8.80e+03 6.60e+03 5.10e+03
 6.10e+04 6.60e+04 7.90e+01 8.60e+03 3.70e+03 1.10e+02 4.40e+04 6.95e+02
 1.60e+03 6.20e+03 5.30e+04 1.40e+03 3.00e+03 7.20e+03 5.80e+03 3.80e+03
 9.60e+03 4.50e+04 6.30e+04 4.90e+04 7.70e+04 4.40e+03 7.00e+04 9.30e+03
 8.10e+03 3.60e+04 6.90e+03 7.40e+03 8.40e+04 9.70e+04 2.00e+03 1.90e+03
 1.80e+03 5.30e+03 4.70e+04 5.56e+02 5.26e+02 7.60e+04 7.60e+03 5.90e+04
 9.70e+03 7.80e+04 7.20e+04 4.30e+04 7.70e+03 6.30e+03 3.16e+02 9.30e+04
 6.50e+04 7.90e+04 1.00e+05 5.80e+04 5.00e+04 6.80e+04 6.40e+04 3.40e+04
 6.70e+04 6.00e+04 9.40e+04 9.90e+03 2.32e+02 9.90e+04 6.24e+02 9.50e+04
 8.50e+00 4.10e+01 2.92e+02 8.00e+04 1.70e+03 7.40e+04 6.20e+04 6.90e+04
 7.50e+04 9.80e+04 8.50e+04 8.20e+04 9.60e+04 8.70e+04 7.10e+04 8.60e+04
 9.10e+04 8.10e+04 9.20e+04 8.30e+04 8.80e+04 7.04e+02 8.62e+02 8.99e+02
 3.78e+02 4.80e+03 2.66e+02 3.75e+02 1.30e+03 9.75e+02 9.80e+02 4.10e+03
 8.90e+04 6.96e+02 5.44e+02 5.25e+02 9.20e+02 7.79e+02 8.53e+02 7.20e+02
 7.13e+02 7.72e+02 3.18e+02 5.80e+01 2.41e+02 1.95e+02 8.57e+02 5.10e+01
 9.53e+02 8.65e+02 2.51e+02 9.30e+02 5.60e+02 3.13e+02 7.46e+02 2.03e+02
 2.60e+01 3.16e+02 2.39e+02 3.71e+02 2.20e+02 7.30e+02 7.56e+02 9.10e+01
 2.93e+02 1.70e+01 7.60e+01 1.60e+01 3.17e+02 7.80e+01 9.24e+02 8.18e+02
 8.10e+01 9.39e+02 1.69e+02 4.50e+01 9.65e+02 9.00e+04 5.45e+02 6.10e+01
 2.83e+02 6.55e+02 7.14e+02 9.30e+01 8.72e+02 1.21e+02 3.22e+02 9.76e+02
 2.06e+02 9.54e+02 4.44e+02 7.17e+02 2.10e+02 6.09e+02 3.08e+02 3.06e+02
 1.75e+02 3.50e+02 3.83e+02 4.54e+02 1.00e+03 7.00e+01 8.12e+02 4.42e+02
 8.42e+02 4.17e+02 4.12e+02 4.59e+02 4.78e+02 3.35e+02 7.82e+02 7.21e+02
 4.30e+02 4.20e+02 1.92e+02 4.60e+02 7.28e+02 4.95e+02 8.16e+02 4.14e+02
 5.05e+02 8.87e+02 6.13e+02 7.78e+02 6.83e+02 5.92e+02 1.85e+02 8.60e+02
 6.47e+02 3.73e+02 4.37e+02 5.98e+02 7.16e+02 5.85e+02 9.82e+02 2.19e+02
 5.50e+01 3.23e+02 6.91e+02 5.11e+02 9.51e+02 9.63e+02 2.50e+01 5.54e+02
 3.51e+02 2.70e+01 8.20e+01 2.08e+02 5.51e+02 2.90e+01 1.03e+02 1.16e+02
 1.53e+02 2.09e+02 4.99e+02 1.73e+02 5.97e+02 8.05e+02 1.22e+02 4.11e+02
 4.00e+02 8.01e+02 7.87e+02 5.00e+01 6.43e+02 9.85e+02 5.16e+02 8.37e+02
 7.00e+02 2.00e+01 4.98e+02 6.00e+02 5.56e+02 2.21e+02 2.28e+02 1.76e+02
 3.40e+01 2.59e+02 1.64e+02 4.58e+02 6.29e+02 2.80e+01 2.88e+02 7.75e+02
 7.85e+02 6.36e+02 9.16e+02 9.94e+02 3.09e+02 4.85e+02 9.14e+02 9.03e+02
 6.80e+02 5.00e+02 5.40e+01 5.62e+02 8.47e+02 9.48e+02 8.11e+02 2.70e+02
 4.80e+01 5.23e+02 7.84e+02 2.80e+02 2.40e+01 8.92e+02 1.54e+02 1.80e+01
 3.30e+01 8.60e+02 3.64e+02 3.87e+02 6.26e+02 1.61e+02 8.79e+02 3.90e+01
 1.70e+02 1.41e+02 1.60e+02 1.44e+02 1.43e+02 1.90e+02 3.76e+02 1.93e+02
 4.73e+02 2.46e+02 7.30e+01 2.53e+02 9.57e+02 4.20e+02 7.20e+01 4.04e+02
 4.70e+02 2.26e+02 2.40e+02 8.90e+01 2.34e+02 2.57e+02 8.61e+02 4.67e+02
 6.76e+02 5.52e+02 5.82e+02 6.19e+02]
```

Replacing nan with median

Replacing Nans with median

```
[11]: df['Size'] = df['Size'].fillna(df['Size'].median())

[12]: print(df['Size'].isna().sum())
```

```
0
```

Checking if size is numeric

```
[13]: #Checking if size is numeric
      df['Size'].dtype
```

```
[13]: dtype('float64')
```

4.2 Converting reviews to Numeric

```
[14]: #Converting Reviews to numeric
      df['Reviews'] = df['Reviews'].astype(float)

[15]: df['Reviews'].dtype
```

```
[15]: dtype('float64')
```

4.3 Fixing Installs

Fixing Installs

```
[16]: #Replacing '+', ',' and converting to integer
      df['Installs'] = df['Installs'].str.replace('+', '', regex=False).str.replace(',', '', regex=False).astype(int)

[17]: print(df['Installs'].head())
      print(df['Installs'].dtype)
```

```
0       10000
1      500000
2     5000000
3    50000000
4      100000
Name: Installs, dtype: int64
int64
```

4.4 Fixing Price

```
[18]: #removing '$' and converting to numeric
      df['Price'] = df['Price'].str.replace('$', '', regex=False).astype(float)

[19]: print(df['Price'].head())
      print(df['Price'].dtype)
```

```
0    0.0
1    0.0
2    0.0
3    0.0
4    0.0
Name: Price, dtype: float64
float64
```

Checking data types

```
[20]: df.dtypes
```

```
[20]: App              object
      Category         object
      Rating          float64
      Reviews         float64
      Size            float64
      Installs          int64
      Type             object
      Price           float64
      Content Rating   object
      Genres           object
      Last Updated     object
      Current Ver      object
      Android Ver      object
      dtype: object
```

4.5 Sanity Checks

```
[21]: #Rating should be between 1 and 5, dropping rows outside the range
      df = df.drop(df[df['Rating'] > 5].index)
      df = df.drop(df[df['Rating'] < 1].index)

[22]: #Reviews should not be greater than installs, only review if installed
      df = df[df['Reviews'] <= df['Installs']]

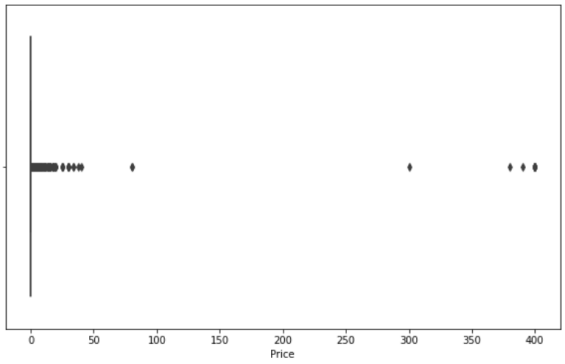
[23]: #for type=free, price should be 0
      df = df.drop(df[(df['Type'] == 'Free') & (df['Price'] > 0)].index)

[24]: df = df.reset_index(drop=True)
```

5. Univariate analysis
   - Price Boxplot

```
#box plot for price
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(10, 6))
sns.boxplot(x=df['Price'])
```

```
<AxesSubplot: xlabel='Price'>
```

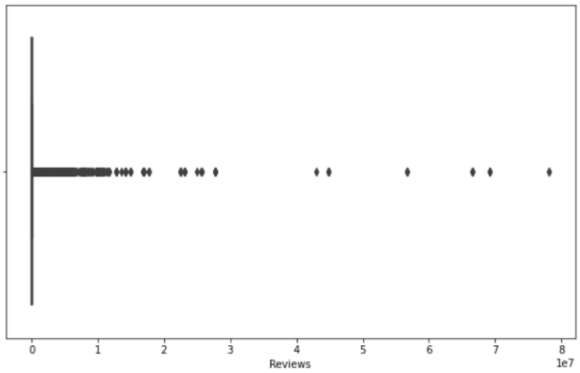


Most apps are priced at or near 0, with the median also being close to zero. However, there are significant outliers with prices reaching up to $400, suggesting a small number of unusually expensive apps that may be junk or incorrectly

   - Reviews Boxplot

```
[26]: _#Reviews Boxplot
      plt.figure(figsize=(10, 6))
      sns.boxplot(x=df['Reviews'])
```

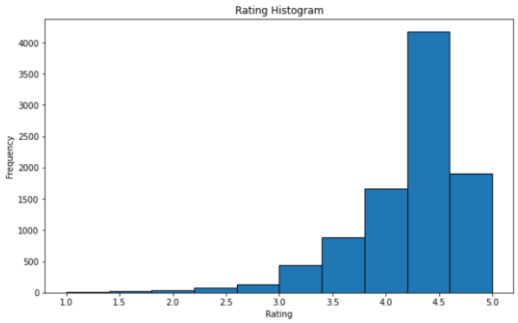
```
[26]: <AxesSubplot: xlabel='Reviews'>
```



Most apps have reviews clustered very close to 0. Outliers are scattered between approximately 10 million and 40 million, with the most extreme outlier sitting at around 40 million reviews.

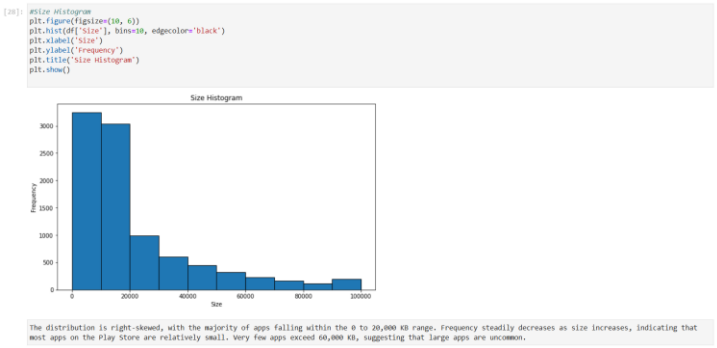
   - Ratings Histogram

```
[27]: #Rating Histogram
plt.figure(figsize=(10, 6))
plt.hist(df['Rating'], bins=10, edgecolor='black')
plt.xlabel('Rating')
plt.ylabel('Frequency')
plt.title('Rating Histogram')
plt.show()
```



Ratings are heavily concentrated between 4.0 and 5.0, suggesting users generally rate apps positively on the Play Store. Very few apps receive low ratings.

- Histogram for Size



6. Outlier treatment

   Dropping suspicious prices

Checking out the records with very high prices (>'$200).

```
[29]: print("Apps priced at $200 or more:")
display(df[df['Price'] >= 200][['App', 'Price', 'Category']])
```

Apps priced at $200 or more:

	App	Price	Category
4035	most expensive app (H)	399.99	FAMILY
4188	💎 I'm rich	399.99	LIFESTYLE
4193	I'm Rich - Trump Edition	400.00	LIFESTYLE
5040	I am rich	399.99	LIFESTYLE
5043	I am Rich Plus	399.99	FAMILY
5044	I am rich VIP	299.99	LIFESTYLE
5045	I Am Rich Premium	399.99	FINANCE
5046	I am extremely Rich	379.99	LIFESTYLE
5047	I am Rich!	399.99	FINANCE
5048	I am rich(premium)	399.99	FINANCE
5051	I Am Rich Pro	399.99	FAMILY
5053	I am rich (Most expensive app)	399.99	FINANCE
5055	I Am Rich	389.99	FAMILY
5058	I am Rich	399.99	FINANCE
5062	I AM RICH PRO PLUS	399.99	FINANCE

From the boxplot, most apps are priced between $0 and $50, making $200 an unusually high price. Upon inspection, apps priced above $200 appear to be junk or incorrectly priced apps that do not represent real Play Store pricing. The names of the apps also

suggest they are junk apps. These records will be dropped to prevent them from skewing the model.

Dropping prices above $200

```
[30]: #drop prices above $200
      df = df.drop(df[df['Price'] >= 200].index)
```

```
[31]: #Max price after dropping
      print("Max price after dropping:", df['Price'].max())
```

```
Max price after dropping: 79.99
```

Dropping apps that have more than 2 000 000 reviews

```
[32]: #Drop Apps that have more than 2 000 000 reviews
      df = df.drop(df[df['Reviews'] > 2000000].index)
```

```
[33]: #Max reviews after dropping
      print("Max reviews after dropping:", df['Reviews'].max())
```

```
Max reviews after dropping: 1986068.0
```

Dropping apps with more than 10 000 000 Installs

```
#Find out the different percentiles of Installs
print("Installs Percentiles:")
print(df['Installs'].quantile([0.1, 0.25, 0.5, 0.7, 0.9, 0.95, 0.99]))
```

```
Installs Percentiles:
0.10         1000.0
0.25        10000.0
0.50       500000.0
0.70      1000000.0
0.90     10000000.0
0.95     10000000.0
0.99    100000000.0
Name: Installs, dtype: float64
```

The 90th and 95th percentiles share the same value of 10 000 000 installs,indicating a large cluster of Apps at 10M installs, Beyond this point, there is a siginificant jump to 100M installs at the 99th percentile, suggesting that the top 4% values are widely spread outliers, therefore the 95th percentile is selected as the uppper cut off to remove these outliers.

```
#Dropping apps that have more than 10 000 000 installs
df = df.drop(df[df['Installs'] > 10000000].index)
```

```
#Display Max number of installs after dropping
print("Max installs after dropping:", df['Installs'].max())
```

```
Max installs after dropping: 10000000
```

Resetting index after dropping

```
[37]: #Reset index
      df = df.reset_index(drop=True)
```

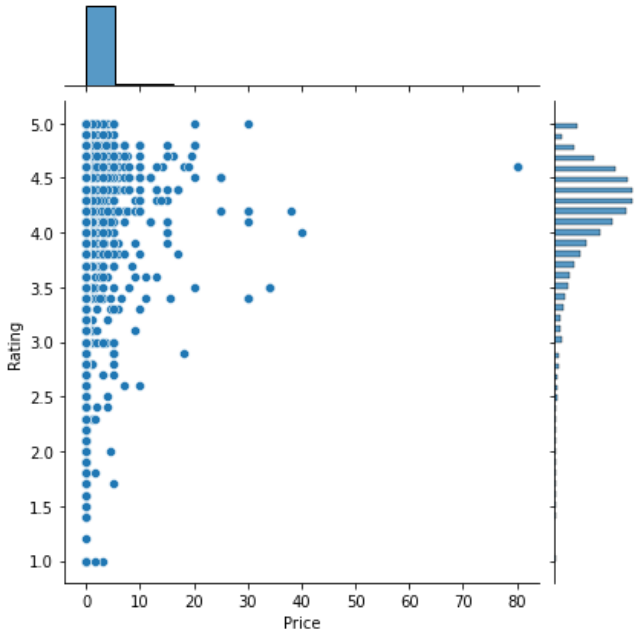
```
[38]: print("Outlier treatment complete. Current shape of dataset:", df.shape)
```

```
Outlier treatment complete. Current shape of dataset: (8496, 13)
```

7. Bivariate Analysis

Price vs Rating join plot

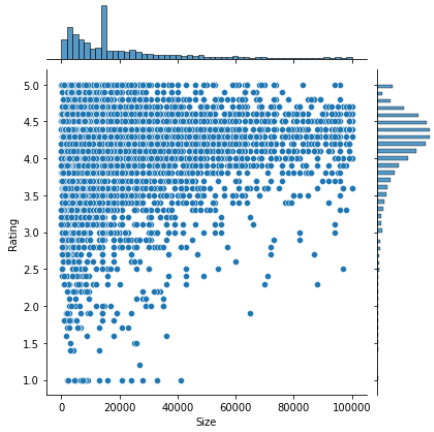
```
# Rating vs Price joinplot
sns.jointplot(y='Rating', x='Price', data=df, kind='scatter')
```



Most apps are clustered at low prices ($0-$10), with ratings spread across the full range (1-5). There is no clear pattern, and rating does not increase with price. Higher priced apps do not necessarily receive better ratings.

Size vs Rating Join plot

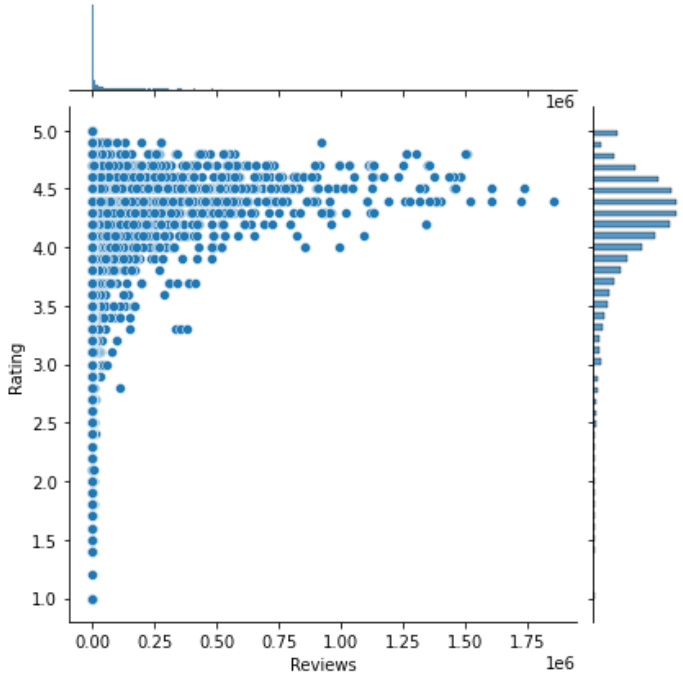
```
#size vs rating joinplot
sns.jointplot(y='Rating', x='Size', data=df, kind='scatter')
```



Heavier apps are not necessarily rated better. Apps are spread across the full-size range (0 - 100,000 KB), with most apps regardless of size rated 3.5 and above, indicating that both light and heavy apps receive high ratings. Size does not appear to have a significant influence on rating.

Review Vs Rating Joinplot

```
#Reviews vs Rating joinplot
sns.jointplot(y='Rating', x='Reviews', data=df, kind='scatter')
```

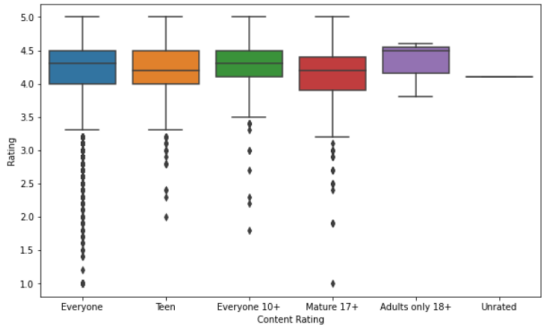


Rating and reviews show that more reviews do not always mean a better rating. While apps with high review counts tend to score between 4.0 and 5.0, the highest ratings are overwhelmingly clustered among apps with very few reviews, showing that high ratings are frequently achieved without high review counts.

Rating vs Content Rating Boxplot

```
#Rating vs Content Rating boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(x='Content Rating', y='Rating', data=df)
```

```
<AxesSubplot: xlabel='Content Rating', ylabel='Rating'>
```



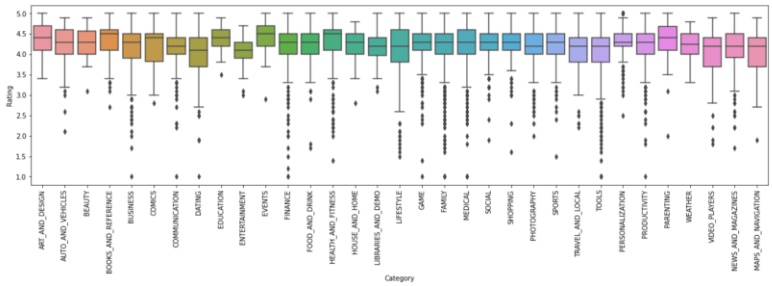
'Adults only 18+' category appears to be liked better with the highest median score, no low outliers and no ratings lower than 4

'Adults only 18+' category appears to be liked better with the highest median score, no low outliers and no ratings lower than 4

Category Vs Rating Boxplot

```
#Rating vs Category boxplot
plt.figure(figsize=(20, 5))
plt.xticks(rotation=90)
sns.boxplot(x='Category', y='Rating', data=df)
```

```
<AxesSubplot: xlabel='Category', ylabel='Rating'>
```



The Events and Health & Fitness categories appear to have the best ratings overall because their median ratings are slightly higher than those of the other categories. However, Events appears to stand out slightly, as its box is positioned at the top of the rating scale, suggesting that apps in this category are generally rated very positively.

The Events and Health & Fitness categories appear to have the best ratings overall because their median ratings are slightly higher than those of the other categories. However, Events appears to stand out slightly, as its box is positioned at the top of the rating scale, suggesting that apps in this category are generally rated very positively.

8. Data Processing

Creating a new dataframe

```
#create a new dataframe : inp1
inp1 = df.copy()
```

Applying log transformation to Installs and Reviews to prevent their large values from skewing the model.

```
#Apply log transformation to Reviews and Installs
inp1['Reviews'] = np.log1p(inp1['Reviews'])
inp1['Installs'] = np.log1p(inp1['Installs'])
```

Dropping Columns that are not useful

```
#Dropping App, Last Updated, Current Ver  , and Android ver
inp1 = inp1.drop(['App', 'Last Updated', 'Current Ver', 'Android Ver'], axis=1)
```

```
inp1.head()
```

	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres
0	ART_AND_DESIGN	4.1	5.075174	19000.0	9.210440	Free	0.0	Everyone	Art & Design
1	ART_AND_DESIGN	3.9	6.875232	14000.0	13.122365	Free	0.0	Everyone	Art & Design;Pretend Play
2	ART_AND_DESIGN	4.7	11.379520	8700.0	15.424949	Free	0.0	Everyone	Art & Design
3	ART_AND_DESIGN	4.3	6.875232	2800.0	11.512935	Free	0.0	Everyone	Art & Design;Creativity
4	ART_AND_DESIGN	4.4	5.123964	5600.0	10.819798	Free	0.0	Everyone	Art & Design

Dummy Encoding

```
#Dummy encoding for Category,type, content rating, and genres
inp2 = pd.get_dummies(inp1, columns=['Category', 'Genres', 'Content Rating', 'Type'], drop_first=True).astype(np.int64)
```

```
inp2.head()
```

	Rating	Reviews	Size	Installs	Price	Category_AUTO_AND_VEHICLES	Category_BEAUTY	Category_BOOKS_AND_REFERENCE	Category_BUSINESS	Category_COMICS	...	Genres_Video Players & Editors;Creativity	Genres_Video Players & Editors;Music & Video	Genre
0	4	5	19000	9	0	0	0	0	0	0	...	0	0	
1	3	6	14000	13	0	0	0	0	0	0	...	0	0	
2	4	11	8700	15	0	0	0	0	0	0	...	0	0	
3	4	6	2800	11	0	0	0	0	0	0	...	0	0	
4	4	5	5600	10	0	0	0	0	0	0	...	0	0	

5 rows × 157 columns

9. 70 – 30 train-test split

```
#Train-test split 70-30
from sklearn.model_selection import train_test_split

df_train, df_test = train_test_split(inp2, test_size=0.3, random_state=0)

print("Train shape:", df_train.shape)
print("Test shape:", df_test.shape)
```

```
Train shape: (5947, 157)
Test shape: (2549, 157)
```

10. Separate in X and Y

```
#Separate into X and Y
X_train = df_train.drop('Rating', axis=1)
y_train = df_train['Rating']
X_test = df_test.drop('Rating', axis=1)
y_test = df_test['Rating']
```

```
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (5947, 156)
y_train shape: (5947,)
X_test shape: (2549, 156)
y_test shape: (2549,)
```

11. Model Building

Model Building

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
#R2 on the train set
y_train_pred = lr.predict(X_train)
r2_train = r2_score(y_train, y_train_pred)
print("R2 on the train set:", r2_train)
```

```
R2 on the train set: 0.10717317394606929
```

```
#R2 on the test set
y_test_pred = lr.predict(X_test)
r2_test = r2_score(y_test, y_test_pred)
print("R2 on the test set:", r2_test)
```

```
R2 on the test set: 0.08741956828875186
```